

# PART 06

---

## 언어모델과 에이전트

- 화학·단백질 언어모델 · RAG · LLM 에이전트 실습 -

## 과정 개요 · 학습 목표 · 실습 환경

### 1) 과정 개요

본 실습은 언어모델(LLM)의 개념을 신약개발 도메인에 직접 적용해 본다. 화학 언어모델(MoLT5)로 분자와 텍스트를 상호 변환하고, 단백질 언어모델(ESM-2 / ESM-3)로 결합 펩타이드를 설계·최적화하며, RAG 로 문서 근거에 기반한 답변을 생성한다. 마지막으로 여러 LLM 에이전트를 조합한 Co-Scientist 와, 계획→검증→보고서를 자동 수행하는 Claude Code 에이전트 하네스(PDE5 저해제 탐색)를 다룬다.

### 2) 학습 목표

- 화학 언어모델(MoLT5)로 텍스트→SMILES, SMILES→텍스트 양방향 변환을 수행한다.
- 단백질 언어모델 ESM-2(masked LM)로 표적 결합 펩타이드를 생성하고 in silico 방향진화로 최적화한다.
- 생성형 단백질 언어모델 ESM-3 로 서열 채우기(inpainting)와 구조 예측을 수행한다.
- RAG(검색-증강 생성) 파이프라인(문서 분할→임베딩→검색→LLM 생성)을 처음부터 구축한다.
- 다중 에이전트 Co-Scientist(LangGraph)로 표적 후보를 자동 평가·순위화한다.
- 무-날조·단계별 검증 게이트를 갖춘 에이전트 하네스의 설치·운영 원리를 이해한다.

### 3) 실습 환경

- 주 실습 = GitHub 저장소 → Google Colab: 각 노트북을 Colab 배지로 열어 직접 실행한다.
    - ▷ **GitHub 저장소** : fourmodern/2026\_aidrugdiscovery (폴더: Day06\_LLM\_Agent/)
    - ▷ **Colab URL 패턴** :  
[https://colab.research.google.com/github/fourmodern/2026\\_aidrugdiscovery/blob/main/Day06\\_LLM\\_Agent/<파일>.ipynb](https://colab.research.google.com/github/fourmodern/2026_aidrugdiscovery/blob/main/Day06_LLM_Agent/<파일>.ipynb)
    - ▷ **런타임** : Colab 의 [런타임] > [런타임 유형 변경]에서 GPU(T4) 선택 권장 (ESM/RAG 가속)
  - 에이전트 하네스(6 절)는 Claude Code 에서 폴더를 여는 방식 — 설치하는 함께, 탐색·보고서 실행은 강사 데모 (B 방식).
  - 필요 API 키(해당 실습에서만):
    - ▷ **Google Gemini API** : Co-Scientist(5 절). Google AI Studio 에서 발급  
<https://aistudio.google.com/app/apikey>
    - ▷ **Hugging Face 로그인/동의** : RAG 의 google/gemma-2b-it 등 gated 모델 사용 시 라이선스 동의 필요
- ※ 결과물은 모두 '가설'이며, 실제 검증(합성·효소 어세이·실험)은 별도임을 유의한다.

| 절 | 주제                   | 노트북 / 폴더                      | 핵심 도구                               |
|---|----------------------|-------------------------------|-------------------------------------|
| 1 | 실습 환경 준비             | (공통)                          | GitHub, Colab, API                  |
| 2 | 화학 언어모델 MoLT5        | t042_molt5.ipynb              | transformers, RDKit                 |
| 3 | 단백질 LM ESM-2 / ESM-3 | t110_esm2...,<br>t111_esm3... | transformers, esm,<br>evo_prot_grad |
| 4 | LLM 활용 & RAG         | t050_simple-local-rag.ipynb   | sentence-transformers,<br>Gemma     |
| 5 | LLM 에이전트 Co-         | T004_nslc_llm_coscien         | LangGraph, Gemini                   |

|   |                    |               |                            |
|---|--------------------|---------------|----------------------------|
|   | Scientist          | tist.ipynb    |                            |
| 6 | [데모] PDE5 에이전트 하네스 | pde5_harness/ | Claude Code, RDKit, ChEMBL |

## 1. 실습 환경 준비 (GitHub → Colab)

### 1) 실습 개요

- 과제 : GitHub 저장소의 Day06 노트북을 Colab 에서 여는 방법과 런타임(GPU) · API 키 준비 절차를 익힌다.

### 2) 실습 사이트

#### ▷ 참조사이트

[https://github.com/fourmodern/2026\\_aidrugdiscovery](https://github.com/fourmodern/2026_aidrugdiscovery)

<https://colab.research.google.com>

#### ▷ 실습사이트 (Colab 배지 URL — 각 절 노트북)

[https://colab.research.google.com/github/fourmodern/2026\\_aidrugdiscovery/blob/main/](https://colab.research.google.com/github/fourmodern/2026_aidrugdiscovery/blob/main/Day06_LLM_Agent/t042_molt5.ipynb)

[Day06\\_LLM\\_Agent/t042\\_molt5.ipynb](#)

...(파일명만 t110\_esm2\_peptide\_optimization\_tutorial / t111\_esm3\_protein\_design / t050\_simple-local-rag / T004\_nslc\_llm\_coscientist 로 교체)

### Step-by-Step 구축 가이드

#### ▷ 저장소 열기

작업 내용 : GitHub 에서 저장소를 확인하고 Day06\_LLM\_Agent 폴더의 노트북 목록을 본다.

#### ▷ Colab 으로 노트북 열기

작업 내용 : 노트북 상단의 'Open In Colab' 배지를 클릭하거나, 위 Colab URL 패턴에 파일명을 넣어 접속한다.

#### ▷ 런타임(GPU) 설정

작업 내용 : 상단 메뉴에서 GPU 런타임으로 변경한다.

#### Configure

[런타임] > [런타임 유형 변경] > 하드웨어 가속기 = T4 GPU > 저장

#### ▷ 셀 실행

작업 내용 : 각 코드 셀을 위에서 아래로 순서대로 실행한다(Shift+Enter). 첫 셀은 대개 라이브러리 설치이다.

#### ▷ API 키 입력(해당 실습만)

작업 내용 : Co-Scientist 는 실행 중 Gemini API 키를 입력받는다. 키는 노트북에 저장하지 말고 런타임마다 입력한다.

#### [1-1] Colab 에서 노트북 GitHub 최신본으로 열기(URL 직접 접속)

```
# 브라우저 주소창에 아래 형식으로 접속하면 GitHub 의 최신 노트북이 Colab 으로 열립니다.  
https://colab.research.google.com/github/fourmodern/2026\_aidrugdiscovery/blob/main/Day06\_LLM\_Agent/t042\_molt5.ipynb
```

#### [1-2] GPU 사용 여부 확인(첫 셀 실행 후)

```
import torch  
print('CUDA 사용 가능:', torch.cuda.is_available())  
print('장치:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')
```

예상 출력 : CUDA 사용 가능: True / 장치: Tesla T4 (GPU 런타임일 때). CPU 런타임이면 False 로 출력된다.

## 2. 화학 언어모델 실습 — MolT5 (분자 ↔ 텍스트)

### 1) 실습 개요

- 과제 : T5 기반 화학 언어모델 MolT5 로 (1) 텍스트 설명 → SMILES 생성, (2) SMILES → 텍스트 설명 생성, (3) 설명을 변형해 유사 구조를 생성하는 양방향 변환을 실습한다.

MolT5 는 T5(Text-To-Text Transfer Transformer) 아키텍처 기반의 분자 언어모델로, 분자 설명과 SMILES 등 표현 사이의 매핑을 학습한다. 본 실습에서는 PDE5 저해제 sildenafil(비아그라)을 예시 분자로 사용한다.

### 2) 실습 사이트

- ▷ 실습사이트 (Colab)

[https://colab.research.google.com/github/fourmodern/2026\\_aidrugdiscovery/blob/main/Day06\\_LLM\\_Agent/t042\\_molt5.ipynb](https://colab.research.google.com/github/fourmodern/2026_aidrugdiscovery/blob/main/Day06_LLM_Agent/t042_molt5.ipynb)

- ▷ 참조 모델 (Hugging Face)

laituan245/molT5-large-caption2smiles (텍스트→SMILES)

laituan245/molT5-large-smiles2caption (SMILES→텍스트)

### Step-by-Step 구축 가이드

- ▷ 환경 설정 (셀 1)

작업 내용 : transformers 와 RDKit 을 설치한다.

#### [2-1] 라이브러리 설치

```
!pip install -q transformers rdkit
```

- ▷ 라이브러리 가져오기 (셀 2)

#### [2-2] 임포트

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
from rdkit.Chem import Draw
from rdkit import Chem
import pandas as pd
from IPython.display import display
```

- ▷ 모델·토크나이저 로드 (셀 3)

작업 내용 : caption2smiles(설명→SMILES) 모델과 토크나이저를 로드한다.

#### [2-3] MolT5 (caption2smiles) 로드

```
tokenizer = T5Tokenizer.from_pretrained(
    "laituan245/molT5-large-caption2smiles", model_max_length=512)
model = T5ForConditionalGeneration.from_pretrained(
    "laituan245/molT5-large-caption2smiles")
```

- ▷ 텍스트 설명 → 분자 (셀 4)

작업 내용/입력 : 분자에 대한 자연어 설명을 넣어 SMILES 를 생성하고 RDKit 으로 시각화한다.

#### [2-4] 설명 → SMILES 생성 (PDE5 저해제 sildenafil 예시)

```

input_text = ('The molecule is sildenafil, a selective phosphodiesterase '
             'type 5 (PDE5) inhibitor used to treat erectile dysfunction '
             'and pulmonary arterial hypertension.')
inputs = tokenizer.encode(input_text, return_tensors='pt')
output_ids = model.generate(inputs, max_length=512)
smiles = tokenizer.decode(output_ids[0], skip_special_tokens=True)
print('생성된 SMILES:', smiles)

mol = Chem.MolFromSmiles(smiles)
if mol:
    display(Draw.MolToImage(mol, size=(300, 300)))

```

예상 출력: 설명에 대응하는 SMILES 문자열과 2D 분자 이미지. (생성 결과는 실행마다 다를 수 있음 — 확인 권장)

#### ▷ 여러 설명 일괄 처리 (셀 5)

작업 내용: 여러 설명을 DataFrame 으로 만들어 각각 SMILES 를 생성한다(반복 generate).

#### ▷ SMILES → 설명 (셀 7)

작업 내용: 방향을 반대로 하여 smiles2caption 모델로 SMILES 를 자연어 설명으로 변환한다.

### [2-5] SMILES → 설명 생성 (sildenafil SMILES 입력)

```

tokenizer_caption = T5Tokenizer.from_pretrained(
    "laituan245/molT5-large-smiles2caption", model_max_length=512)
model_caption = T5ForConditionalGeneration.from_pretrained(
    "laituan245/molT5-large-smiles2caption")

# sildenafil(비아그라)의 SMILES
smiles_example = "CCCC1nn(C)c2c1nc([nH]c2=O)-c1cc(ccc1OCC)S(=O)(=O)N1CCN(C)CC1"
inputs = tokenizer_caption.encode(smiles_example, return_tensors='pt')
output_ids = model_caption.generate(inputs, max_length=512)
print('생성된 설명:', tokenizer_caption.decode(output_ids[0],
                                              skip_special_tokens=True))

```

#### ▷ 설명 변형 → 유사 구조 (셀 8)

작업 내용: SMILES 에서 얻은 설명에 구조 변형 문장(예: 피페라진 N-메틸기를 에틸기로 치환)을 덧붙인 뒤 다시 caption2smiles 로 변형 분자를 생성하고, 원본과 나란히 시각화한다.

정리: MolT5 는 텍스트와 분자 표현을 오가며 반응 예측·물성 추정 등 화학정보학 작업에 활용할 수 있다. 생성된 구조는 유효성(RDKit 파싱)과 화학적 타당성을 반드시 검토해야 한다.

### 3. 단백질 언어모델 실습 — ESM-2 / ESM-3

#### 1) 실습 개요

- 과제 : (3-1) ESM-2 기반 masked LM(PepMLM)으로 표적 단백질 결합 펩타이드를 생성하고 EvoProtGrad로 in silico 방향진화(최적화)한다. (3-2) 생성형 멀티모달 모델 ESM-3로 서열 채우기(inpainting)와 구조 예측을 수행한다.

예시 표적은 PDE5A(phosphodiesterase type 5A; UniProt O76074, 875 aa) — PDE5 저해제(sildenafil, tadalafil)의 표적이다.

| 구분    | ESM-2 (t110)           | ESM-3 (t111)           |
|-------|------------------------|------------------------|
| 성격    | masked language model  | 생성형 멀티모달               |
| 다루는 것 | 서열만                    | 서열 + 구조 + 기능           |
| 사용 방식 | EsmForMaskedLM 로짓 스코어링 | model.generate() 반복 생성 |
| 구조 예측 | 불가                     | 가능 (좌표 + pTM/pLDDT)    |

#### 2) 실습 사이트

##### ▷ 실습사이트 (Colab)

.../Day06\_LLM\_Agent/t110\_esm2\_peptide\_optimization\_tutorial.ipynb

.../Day06\_LLM\_Agent/t111\_esm3\_protein\_design.ipynb

##### ▷ 참조 모델

ChatterjeeLab/PepMLM-650M (ESM-2 계열), facebook/esm2\_t6\_8M\_UR50D,  
EvolutionaryScale/esm3-sm-open-v1

#### Step-by-Step 구축 가이드 (3-1) ESM-2 펩타이드 생성·최적화

##### ▷ 설치 (셀 1)

작업 내용 : transformers 만 설치한다. (PyPI의 'esm' 패키지는 ESM-3이며 여기선 불필요. torch는 Colab 기본 제공)

##### [3-1] 설치

```
!pip install -q transformers
```

##### ▷ 모델 로드 (셀 2)

##### [3-2] PepMLM(ESM-2 계열) 로드

```
from transformers import AutoTokenizer, AutoModelForMaskedLM
import torch, pandas as pd, numpy as np
from torch.distributions import Categorical

model = AutoModelForMaskedLM.from_pretrained("ChatterjeeLab/PepMLM-650M")
tokenizer = AutoTokenizer.from_pretrained("ChatterjeeLab/PepMLM-650M")
```

##### ▷ 표적 서열 준비 (셀 3)

작업 내용/입력 : 표적 PDE5A(UniProt O76074)의 아미노산 서열을 토큰화한다.



### [3-3] 표적 단백질 서열 토큰화 (서열은 노트북에 전체 수록)

```
protein_seq = "MERAGPSFGQQRQQQ...GESGQAKRN" # PDE5A, 875 aa (UniProt 076074)
inputs = tokenizer(protein_seq, return_tensors="pt")
```

#### ▷ 펩타이드 생성 (셀 4~6)

작업 내용 : 마스크 토큰(<mask>)을 이어붙인 입력으로부터 top-k 샘플링으로 결합 후보 펩타이드를 생성하고, 각 후보의 pseudo-perplexity(PPL, 낮을수록 좋음)를 계산해 순위화한다.

### [3-4] 결합 펩타이드 생성 실행

```
results_df = generate_peptide(protein_seq, peptide_length=15,
                              top_k=3, num_binders=5)
print(results_df)
```

예상 출력 : 생성 펩타이드 서열과 PPL 값이 담긴 표(낮은 PPL 우선). 생성 결과는 실행마다 달라진다.

#### ▷ in silico 방향진화 (EvoProtGrad, 셀 7~8)

작업 내용 : EvoProtGrad 로 표적 영역을 보존(preserved\_regions)한 채 펩타이드 구간만 변이시켜 결합 적합도를 개선한다. 작은 ESM-2(esm2\_t6\_8M) 전문가 모델과 mutant\_marginal 스코어링을 사용한다.

### [3-5] EvoProtGrad 설치 주의(따옴표 + --no-deps)

```
# ">"가 셀 리다이렉션으로 해석되지 않도록 따옴표로 묶습니다.
# --no-deps 로 현재 transformers 버전을 유지합니다.
!pip install -q --no-deps "evo_prot_grad>=0.2"
```

핵심 설정 : parallel\_chains, n\_steps=50, max\_mutations=15, scoring\_strategy='mutant\_marginal', preserved\_regions=[표적 구간, 링커 구간]. 표적:펩타이드는 ':'로 연결하고 내부적으로 G×20 링커로 치환한다.

## Step-by-Step 구축 가이드 (3-2) ESM-3 서열 설계·구조 예측

#### ▷ 설치 (셀 1)

작업 내용 : esm 패키지를 --no-deps 로 설치(transformers 다운그레이드 방지)하고, 런타임 의존성만 추가 설치한다.

### [3-6] ESM-3 설치

```
!pip install -q --no-deps esm
!pip install -q zstd msgpack-numpy cloudpathlib tenacity pygtrie pydssp brotli
!pip install -q einops "biotite>=1.0.0" py3Dmol
```

#### ▷ 모델 로드 (셀 2)

### [3-7] ESM-3 (esm3\_sm\_open\_v1, 1.4B) 로드

```
import torch
from esm.models.esm3 import ESM3
from esm.sdk.api import ESMProtein, GenerationConfig
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 가중치는 Hugging Face에서 내려받음 (약 2.8GB, CPU에서도 동작)
model = ESM3.from_pretrained("esm3_sm_open_v1", device=device)
```

#### ▷ 서열 채우기 (inpainting, 셀 3)

작업 내용/입력 : '\_'로 마스크한 구간을 ESM-3 가 설계해 채운다(루프/링커 재설계에 대응). track='sequence', num\_steps, temperature 로 제어한다.

### [3-8] 마스크 구간 서열 설계

```
scaffold = "MKTVRQERLKSIVRIL"
tail = "ERLKSIVRILERSKEPVSGAQLAEELSVSRQVIVQDIAYLRSLGYNIVATPRGYVLAGG"
masked_sequence = scaffold + "_" * 12 + tail

prompt = ESMProtein(sequence=masked_sequence)
designed = model.generate(
    prompt,
    GenerationConfig(track="sequence", num_steps=8, temperature=0.7))
print(designed.sequence)
```

#### ▷ 구조 예측 (셀 4)

작업 내용 : 같은 모델에 track='structure'를 요청해 3D 좌표와 신뢰도(pTM, pLDDT)를 얻는다. pLDDT $\geq$ 0.9  
매우 신뢰, pTM $\geq$ 0.5 면 대체로 올바른 fold.

### [3-9] 서열 → 구조 예측

```
structure_prompt = ESMProtein(sequence=designed.sequence)
predicted = model.generate(
    structure_prompt, GenerationConfig(track="structure", num_steps=8))
print("pTM : ", float(predicted.ptm))
print("pLDDT : ", float(predicted.plddt.mean()))
```

#### ▷ 구조 시각화 (셀 5)

작업 내용 : to\_pdb\_string()으로 PDB 텍스트를 얻어 py3Dmol 에 올리고, pLDDT(B-factor)로 색을 칠한다.  
한계 : esm3\_sm\_open\_v1 은 공개 소형(1.4B) 모델이며 대형(98B)은 Forge API 로만 접근 가능. 예측 구조는 모델 추정이므로 실제 검증(발현·결합·열안정성)이 필요하다.

## 4. LLM 활용 & RAG 실습 (Retrieval-Augmented Generation)

### 1) 실습 개요

- 과제 : 로컬 RAG 파이프라인을 처음부터 구축한다. PDF 문서를 텍스트로 추출→문장/청크 분할→임베딩 생성/저장→질의 임베딩과 유사도 검색(top-k)→검색 문맥을 프롬프트에 넣어 LLM 이 근거 기반 답변을 생성한다.

RAG 는 관련 정보를 프롬프트에 주입(Augmentation)해 LLM 의 환각을 줄이고 출처를 함께 제공한다. 도메인 적용 예: 질의를 'mechanism of sildenafil as a PDE5 inhibitor...'로 바꾸려면 코퍼스 PDF 도 PDE5 도메인 문헌으로 교체해야 한다.

### 2) 실습 사이트

- ▷ 실습사이트 (Colab)

.../Day06\_LLM\_Agent/t050\_simple-local-rag.ipynb

- ▷ 임베딩 모델 / 생성 LLM

임베딩: sentence-transformers all-mpnet-base-v2 (출력 768 차원)

생성 LLM(택 1): google/gemma-2b-it · gemma-7b-it · Qwen/Qwen3-1.7B · Qwen3-8B (GPU/양자화에 맞게 선택)

### Step-by-Step 구축 가이드

- ▷ ① 문서 처리 & 임베딩 생성

작업 내용 : PDF 를 PyMuPDF(fitz)로 읽어 페이지별 텍스트를 얻고, spaCy 로 문장 분리 후 10 문장 단위 청크로 묶는다. 짧은 청크( $\leq 30$  토큰)는 제거한다.

#### [4-1] PDF 텍스트 추출 + 청크화 핵심

```
import fitz # PyMuPDF
from spacy.lang.en import English

nlp = English(); nlp.add_pipe('sentencizer')
num_sentence_chunk_size = 10 # 10문장 = 1 청크
```

- ▷ ② 청크 임베딩 & 저장

작업 내용 : all-mpnet-base-v2 로 각 청크를 768 차원 벡터로 임베딩하고 CSV 로 저장한다(GPU 가 CPU 보다 훨씬 빠름).

#### [4-2] 청크 임베딩 생성 및 저장

```
from sentence_transformers import SentenceTransformer
embedding_model = SentenceTransformer("all-mpnet-base-v2", device="cuda")

text_chunk_embeddings = embedding_model.encode(
    text_chunks, convert_to_tensor=True)
pd.DataFrame(pages_and_chunks_over_min_token_len).to_csv(
    "text_chunks_and_embeddings_df.csv", index=False)
```

- ▷ ③ 유사도 검색 (Retrieval)

작업 내용/입력 : 질의를 임베딩한 뒤 dot product 로 전체 청크와의 유사도를 구하고 torch.topk 로 상위 k 개를 뽑는다.

#### [4-3] 질의 임베딩 → 상위 k 청크 검색

```
from sentence_transformers import util
import torch

query = "macronutrients functions" # (PDE5 예: sildenafil 기전 질의로 교체 가능)
query_embedding = embedding_model.encode(query, convert_to_tensor=True)
dot_scores = util.dot_score(a=query_embedding, b=embeddings)[0]
top_results = torch.topk(dot_scores, k=5)
```

예상 출력 : 유사도 점수 상위 5 개 청크의 점수·본문·페이지 번호.

#### ▷ ④ 생성 (Augmented Generation)

작업 내용 : 검색된 문맥을 프롬프트에 넣고(apply\_chat\_template) 로컬 LLM(gemma-2b-it 등)으로 답변을 생성한다. gemma 는 gated 모델이라 Hugging Face 라이선스 동의가 필요하다.

#### [4-4] 생성 LLM 로드 및 답변 생성

```
from transformers import AutoTokenizer, AutoModelForCausalLM
model_id = "google/gemma-2b-it" # 또는 Qwen/Qwen3-1.7B
tokenizer = AutoTokenizer.from_pretrained(model_id)
llm_model = AutoModelForCausalLM.from_pretrained(model_id)

# 검색 문맥 + 질문 → dialogue → 프롬프트
prompt = tokenizer.apply_chat_template(
    conversation=dialogue_template, tokenize=False,
    add_generation_prompt=True)
input_ids = tokenizer(prompt, return_tensors='pt').to('cuda')
outputs = llm_model.generate(**input_ids, max_new_tokens=256)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

정리 : RAG 의 답변은 어떤 청크가 근거로 들어갔는지 함께 확인할 수 있어 출처 추적이 가능하다. 도메인 특화 답변을 원하면 코퍼스(PDF)를 해당 도메인 문헌으로 교체한다.

## 5. LLM 에이전트 실습 — Co-Scientist (다중 에이전트)

### 1) 실습 개요

- 과제 : LangGraph 로 다중 LLM 에이전트 파이프라인(Co-Scientist)을 구성해 폐동맥고혈압(PAH) 표적 후보 유전자들을 자동으로 문헌 검색·분석·평가하고 점수화·순위화한다.

에이전트 구성: 문헌(Literature) → 메커니즘(Mechanism) → 임상(Clinical) → 약물(Drug) → 심판(Judge) 순으로 상태(State)를 전달하며 각 유전자를 평가한 뒤, 상위 후보를 토너먼트로 비교한다.

### 2) 실습 사이트

- ▷ 실습사이트 (Colab)

.../Day06\_LLM\_Agent/T004\_nsclc\_llm\_coscientist.ipynb

- ▷ 필요 API

Google Gemini API 키 (<https://aistudio.google.com/app/apikey>) — 실행 중 getpass 로 입력, 노트북에 저장 금지.

### Step-by-Step 구축 가이드

- ▷ ① 설치 & 임포트 (셀 1~2)

작업 내용 : LangChain/LangGraph/Gemini 연동 및 문헌·시각화 패키지를 설치한다. (구 google-generativeai 는 폐지 → google-genai / LangChain 1.x 는 langchain\_core.prompts 사용)

#### [5-1] 설치

```
!pip install -q langchain langchain-google-genai langgraph
!pip install -q biopython xlrd requests matplotlib seaborn plotly pandas numpy
!pip install -q wordcloud networkx beautifulsoup4
```

- ▷ ② Gemini API 설정 (셀 3)

작업 내용/입력 : API 키를 입력받고, 사용 가능한 Gemini 모델을 런타임에 조회해 선택한다.

#### [5-2] Gemini API 키 입력 및 모델 선택

```
from getpass import getpass
import os
os.environ["GOOGLE_API_KEY"] = getpass("Google Gemini API 키를 입력하세요: ")

from google import genai
from langchain_google_genai import ChatGoogleGenerativeAI
# pick_gemini_model(): client.models.list()로 generateContent 지원 모델 선택
```

- ▷ ③ PubMed 문헌 검색 (셀 4)

작업 내용 : 유전자·질병(PAH) 키워드로 PubMed 를 검색하는 클래스를 구성한다(예: search\_gene\_papers('PDE5A')).

- ▷ ④ 에이전트 프롬프트 & 노드 (셀 5)

작업 내용 : literature/mechanism/clinical/drug/judge 5 종 PromptTemplate 과 각 노드 함수를 정의한다. run\_prompt = (prompt | llm) 파이프로 실행한다(LLMChain 대체).

### ▷ ⑤ LangGraph 워크플로우 (셀 6)

작업 내용 : 5 개 노드를 순서대로 연결한 StateGraph 를 구성·컴파일한다.

#### [5-3] LangGraph 워크플로우 구성

```
from langgraph.graph import StateGraph, END

def create_research_workflow():
    workflow = StateGraph(ResearchState)
    for name, fn in [('literature', literature_agent),
                    ('mechanism', mechanism_agent),
                    ('clinical', clinical_agent),
                    ('drug', drug_agent),
                    ('judge', judge_agent)]:
        workflow.add_node(name, fn)
    workflow.set_entry_point('literature')
    workflow.add_edge('literature', 'mechanism')
    workflow.add_edge('mechanism', 'clinical')
    workflow.add_edge('clinical', 'drug')
    workflow.add_edge('drug', 'judge')
    workflow.add_edge('judge', END)
    return workflow.compile()

research_app = create_research_workflow()
```

흐름 : Literature → Mechanism → Clinical → Drug → Judge (논문검색→메커니즘→임상→약물→최종점수)

### ▷ ⑥ 표적 후보 평가 실행 (셀 7)

작업 내용/입력 : PAH 연관 후보 유전자 리스트를 순회하며 워크플로우를 invoke 하고 최종 점수를 수집한다 (문헌 확인 권장).

#### [5-4] 후보 유전자 평가 루프

```
TARGET_GENES = ["PDE5A", "GUCY1A1", "GUCY1B1", "EDNRA", "EDNRB",
                 "PTGIR", "NOS3", "BMPR2", "ACVRL1", "ENG",
                 "GDF2", "KCNK3", "CAV1", "SMAD9"]

for gene in TARGET_GENES:
    initial_state = {'gene_name': gene, ...}
    result = research_app.invoke(initial_state)
    # result['final_score'] 수집
```

### ▷ ⑦ 토너먼트 & 시각화 (셀 8~9)

작업 내용 : 점수 상위 8 개 유전자로 토너먼트(쌍대 비교)를 수행하고, 최종 점수 랭킹 등을 막대/히트맵으로 시각화한다.

주의 : 노트북의 PubMed 초록 일부는 데모용 텍스트로 대체될 수 있고, LLM 점수는 추론 결과이므로 실제 표적 우선 순위 결정에는 원문 문헌·실험 근거 확인이 필요하다(결과=가설).

## 6. [강사 데모] PDE5 저해제 탐색 에이전트 하네스

### 1) 실습 개요

- 과제 : 자연어 목표를 받아 '계획(PLAN) → 단계별 검증 게이트(EXECUTE) → 근거 인용 보고서(REPORT)'까지 자동 수행하는 Claude Code 에이전트 하네스의 구조를 이해하고, 설치하는 함께 진행한 뒤 실제 탐색·보고서 생성은 강사가 시연한다(B 방식).

▷ **표적** : PDE5A (UniProt O76074, ChEMBL CHEMBL1827). 기전: cGMP 분해 억제 → NO-cGMP 경로 혈관확장

▷ **사례 약물** : sildenafil(CHEMBL192) · tadalafil(CHEMBL779) · vardenafil · avanafil

핵심 개념 : 이 하네스는 에이전트가 '무·날조(No-Fabrication) · provenance(출처) · 단계별 검증'을 지키며 파이프라인을 수행하는 방법을 시연한다. 수치·식별자는 도구가 실제 계산·반환한 값만 사용하고, LLM 이 IC50·물성·ID 를 제어내는 것을 금지한다. 결과는 모두 가설이며 신약 발견이 아니다.

### 2) 실습 참조

▷ 참조사이트

[https://github.com/fourmodern/2026\\_aidrugdiscovery](https://github.com/fourmodern/2026_aidrugdiscovery) (폴더: Day06\_LLM\_Agent/pde5\_harness/)

Claude Code 문서: <https://docs.claude.com/en/docs/claude-code>

### 하네스 구성 (폴더 구조)

#### [6-1] pde5\_harness/ 주요 파일

```
pde5_harness/
├── CLAUDE.md          # 에이전트 규약 (워크플로·하드 규칙·검증 기준)
├── README.md          # 개요·실행법
├── requirements.txt    # rdkit, chembl-webresource-client, requests
├── .claude/
│   ├── settings.json  # PostToolUse 혹 등록
│   ├── skills/        # plan-research, target-lookup, chembl-actives,
│   │               # mol-properties, selectivity-check, report-writer
│   └── hooks/         # verify_provenance.py, no_fabrication_guard.py
├── scripts/           # target_lookup / chembl_actives / mol_properties /
│               # selectivity / verify / mol_utils / docking
└── outputs/           # plan.json, report_pde5.md 생성 위치
```

### 워크플로 (CLAUDE.md 규약)

| 단계       | 스킬 / 스크립트                          | 검증 게이트(통과 조건)                                   |
|----------|------------------------------------|-------------------------------------------------|
| 1. PLAN  | plan-research → outputs/plan.json  | objective/questions/tools/criteria/rules 6 키 포함 |
| 2a. 표적   | target-lookup (target_lookup.py)   | accession=O76074 + 'PDE5' 언급 + 기능 서술            |
| 2b. 활성물질 | chembl-actives (chembl_actives.py) | 실 molecule_chembl_id + SMILES RDKit 파싱          |
| 2c. 물성   | mol-properties (mol_properties.py) | MW/logP/QED/SA 등 물리적 sanity 범위                  |

|           |                                           |                                   |
|-----------|-------------------------------------------|-----------------------------------|
| 2d. 선택성   | selectivity-check (selectivity.py)        | 정량 미날조(정성 보고) + PDE6 기전 서술        |
| 3. REPORT | report-writer →<br>outputs/report_pde5.md | 모든 수치가 앞 단계 도구 결과에 존재(근거 없는 수치 0) |

각 단계는 verification.passed=false 이면 다음 단계 진행 금지 — 최대 2 회 재시도 후에도 실패하면 '미확인/플래그'로 표시하고 보고서 한계 섹션에 명시한다. 혹은 Bash 실행 후 provenance/무-날조를 점검하는 경고형(비차단)이다.

## Step-by-Step (설치는 함께, 실행은 강사 데모)

### ▷ ① 저장소 내려받기 (함께)

작업 내용 : 저장소를 clone 하고 하네스 폴더로 이동한다.

#### [6-2] git clone 및 폴더 이동

```
git clone https://github.com/fourmodern/2026_aidrugdiscovery.git
cd 2026_aidrugdiscovery/Day06_LLM_Agent/pde5_harness
```

### ▷ ② 의존성 설치 (함께)

작업 내용 : RDKit·ChEMBL 클라이언트·requests 를 설치한다(미설치 시 스크립트는 수치를 만들지 않고 graceful 폴백).

#### [6-3] requirements 설치

```
pip install -r requirements.txt
```

### ▷ ③ Claude Code 에서 폴더 열기 (함께)

작업 내용 : 하네스 폴더에서 Claude Code 를 실행하면 CLAUDE.md 규약·스킬·훅이 로드된다.

#### [6-4] Claude Code 실행

```
cd 2026_aidrugdiscovery/Day06_LLM_Agent/pde5_harness
claude
```

### ▷ ④ 스크립트 직접 실행으로 동작 확인 (함께, 노트북 없이도 동작)

#### [6-5] 표준 봉투(result/provenance/verification) 확인

```
python scripts/target_lookup.py
python scripts/chembl_actives.py | python scripts/mol_properties.py --stdin
python scripts/selectivity.py
```

예상 출력 : 각 스크립트가 {result, provenance, verification} JSON 봉투를 반환. 네트워크 불가 시 'OFFLINE DEMO ... 실데이터 아님' 표기로 폴백한다.

### ▷ ⑤ 에이전트 자동 수행 (강사 데모)

작업 내용 : 강사가 Claude Code 에 자연어로 지시하면, 에이전트가 PLAN→단계별 EXECUTE(검증 게이트)→REPORT 를 자동 수행하고 outputs/report\_pde5.md 를 생성한다. 매 단계 훅이 provenance·무-날조를 점검한다.

#### [6-6] 강사 데모 — 자연어 지시(Claude Code 프롬프트)

```
"PDE5 저해제 후보 조사해 보고서 써줘"
```



산출물 : outputs/plan.json(연구 계획), outputs/report\_pde5.md(IMRAD 보고서 — 활성물질/물성 표 + 검증 로그 + 한계 + 참고문헌).

### 하드 규칙 (무-날조 · 과학적 엄밀성)

- ▷ 수치는 도구 실제산값만 — LLM 이 IC50 · 물성 · ChEMBL ID · PMID 등 수치 · 식별자를 지어내는 것 금지.
- ▷ 모든 사실 주장에 출처(provenance): ChEMBL ID / UniProt / PMID / DOI. 없으면 '확인 필요'로 표기.
- ▷ 단계별 검증 게이트 통과 후에만 다음 단계로 진행.
- ▷ 불확실은 불확실로 — 도킹 스코어 · LLM 추론은 '가설'. 결과=가설이며 신약 발견이 아님(실검증은 별도).

## 부록. 참고자료 · 한계

### 1) 노트북 · Colab 링크 요약

| 절 | 파일                                            | Colab URL<br>(.../blob/main/Day06_LLM_Agent/ 뒤에 파일명)  |
|---|-----------------------------------------------|-------------------------------------------------------|
| 2 | t042_molt5.ipynb                              | .../t042_molt5.ipynb                                  |
| 3 | t110_esm2_peptide_optimization_tutorial.ipynb | .../<br>t110_esm2_peptide_optimization_tutorial.ipynb |
| 3 | t111_esm3_protein_design.ipynb                | .../<br>t111_esm3_protein_design.ipynb                |
| 4 | t050_simple-local-rag.ipynb                   | .../t050_simple-local-rag.ipynb                       |
| 5 | T004_nsclc_llm_coscientist.ipynb              | .../<br>T004_nsclc_llm_coscientist.ipynb              |
| 6 | pde5_harness/ (폴더)                            | github.com/fourmodern/<br>2026_aidrugdiscovery        |

공통 접두 : [https://colab.research.google.com/github/fourmodern/2026\\_aidrugdiscovery/blob/main/Day06\\_LLM\\_Agent/](https://colab.research.google.com/github/fourmodern/2026_aidrugdiscovery/blob/main/Day06_LLM_Agent/)

### 2) 주요 모델 · 라이브러리

- ▷ 화학 LM: MolT5 (laituan245/molt5-large-\*)
- ▷ 단백질 LM: ESM-2/PepMLM (ChatterjeeLab/PepMLM-650M, facebook/esm2\_t6\_8M\_UR50D), ESM-3 (EvolutionaryScale/esm3-sm-open-v1), EvoProtGrad
- ▷ RAG: sentence-transformers(all-mpnet-base-v2), transformers(gemma-2b-it/Qwen3), PyMuPDF, spaCy
- ▷ 에이전트: LangGraph, langchain-google-genai(Gemini), Claude Code(하네스)
- ▷ 화학정보학: RDKit, chembl-webresource-client

### 3) 한계 및 유의사항

- 생성·예측 결과는 모두 '가설'이다 — 생성 분자/펩타이드/구조/표적 점수는 실험(합성·결합·어세이·구조)으로 검증해야 한다.
- 생성 결과는 실행마다 달라질 수 있으며(temperature·샘플링), SMILES는 RDKit 파싱 등 유효성 확인이 필요하다.
- 일부 모델은 gated(라이선스 동의)이거나 API 키가 필요하다(gemma, Gemini). 키는 노트북에 저장하지 않는다.
- 본 교재의 서열·수치는 노트북·하네스에서 확인한 값이며, 불확실한 항목은 '확인 권장'으로 표기했다.
- PDE5 하네스는 알려진 화학공간의 후보를 선별·정리하는 시연이며 신약 발견이 아니다. 무·날조 원칙(수치=도구 실제산값)을 준수한다.